

5. 구성 속성 사용하기.

'자동-구성' → Spring Application 개발 단축.

Spring Application Context 에서 구성 속성인 Bean의 속성

↳ JVM 시스템 속성, 명령행 인자, 환경변수 등 여러가지 원천 속성 중에서 설정 가능.

5.1 자동-구성 세부 조정.

[스프링에서 과거의 형태로 서로 다르면서 관련 있는 구성이 있음.]

- Bean wiring (빈 연결)

: Spring application context 에서 Bean으로 생성되는

애플리케이션 컴포넌트 및 상호 간에 주입되는 방법 선언하는 구성.

⇒ Spring 컨테이너가 애플리케이션 실행하면서 필요한 객체 생성하고
서로 연결.

1) @Component, @Service, @Repository, @Controller ... 으로 어노테이션으로
등록된 클래스 자동으로 빈 생성.

2) @Autowired, @Inject, @RequiredArgsConstructor ... 이용해 필요한 빈을
다른 빈에 주입 (Inject).

빈: 연결되는 대상, Spring Container: 생성, 관리, 필요 시 연결.

- Property injection (속성 주입)

: 스프링 애플리케이션 컨텍스트에서 빈의 속성 값을 설정하는 구성.

⇒ Spring이 설정 파일에 정의된 값을 Java 객체에서
필드/생성자, setter 메소드에 자동으로 넣음.

⇒ 이 과거 구성은 Spring xml 구성과 Java 기반 구성 모두에서 종종 같은 곳에서
선언.

5.1.1) 스프링 환경 추상화 이해.

바탕)

'스프링 환경 추상화' => 구성 가능한 모든 속성 한 곳에서 관리하는 개념

속성기 은닉 → 속성 필요로 하는 빈이 스프링 자체에서 해당 속성 사용할 수 있게 해줌.

- ① JVM 시스템 속성
- ② 운영체제 환경 변수
- ③ 환경변 인자
- ④ 애플리케이션의 속성 구성 파일.

용어 정리)

Spring Container : Bean을 생성, 주입, 생명주기 관리하는 핵심 개념 전체 의미.

=> 스프링이 중심 역할 전체 가려짐.

=> BeanFactory, ApplicationContext가 스프링 컨테이너의 구현체.

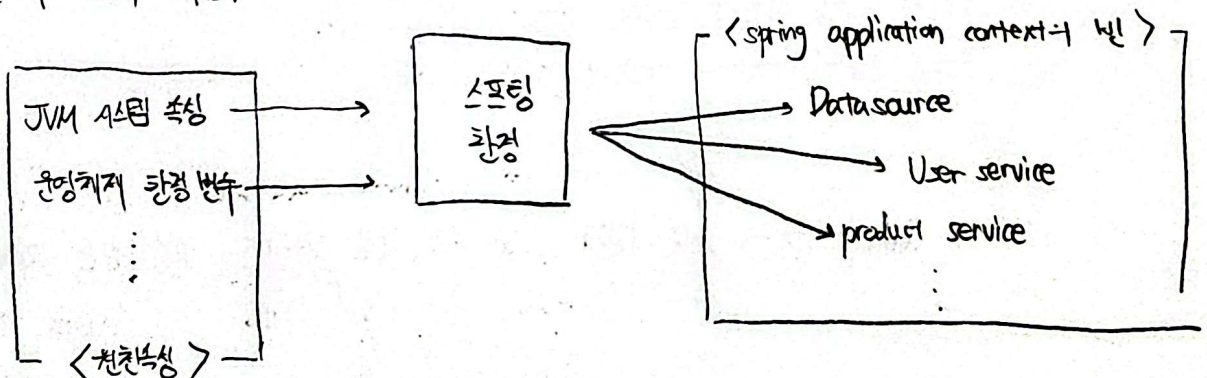
ApplicationContext : 스프링 컨테이너 구현체 중 하나.

=> BeanFactory 확장해 많은 기능 제공

- 메시지 루터
- 이벤트 처리
- 환경 변수 접근
- ...

=> @SpringBootApplication, @ComponentScan
@Configuration ... 전부 ApplicationContext 기반.

스프링 환경에서는 속성들을 한 군데로 모은 후, 각 속성이 주입되는 스프링 빈 사용할 수 있게 해줌.



바깥)

속성의 구현 => 신병 출신지.

Environment => 훈련소 / 행정실.

⋮

이디서 왔든 즉시 부대 배치되기
도와줌

원천 속성 => 개인 병사 정보.

L 이름, 계급, ... => Environment, getProperty("soldier.rank")

5.1.2) 데이터 소스 구성하기.

DataSource => DataSource 빈을 명시적으로 구성.

But, spring boot는 X. => 구성 속성 통해서

일반적인 Java

⋮
ds.setJdbcUrl(" ~ ");
ds.setUsername(" ~ ");
ds.setPassword(" ~ ");
⋮

(하드 코딩)

application.properties

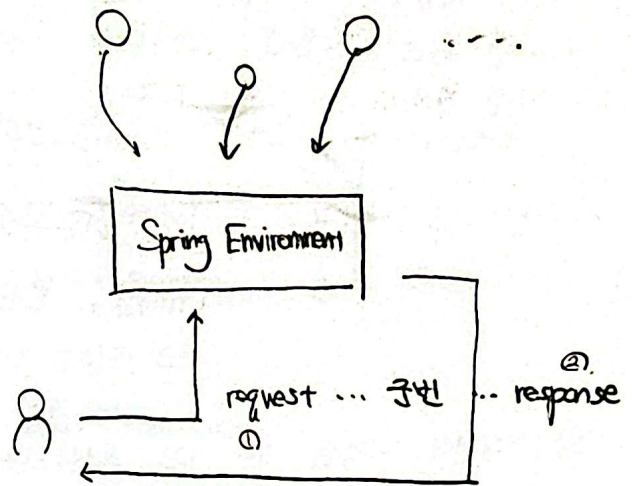
spring.datasource.url = ~
spring.datasource.username = ~
⋮

L 자동으로 DataSource 빈 만들어줌.

① DataSource 빈 자동-구성할 때 spring boot가
이런 속성 설정을 연결 데이터로 사용.

⋮ ; new HikariDataSource() 할 필요 X

② 동적으로 JDBC 커넥션 풀을 classpath에서
자동으로 찾을 수 있다면 DataSource 빈이 그것 사용.
: spring boot는 클래스 경로(classpath)에 어떤 커넥션 풀
라이브러리 존재하는지 확인 후 우선순위에 맞게
자동선택.



- 커넥션 풀을 자동으로 선택하는 방식.

우선순위 { 1 : Hikari CP (기본)
 2 : Tomcat JDBC Connection Pool
 3 : Commons DBCP2

애플리케이션이 시작될 때 DB 커넥션하는 SQL 스크립트의 실행방법.

ex)

SQL 커넥션 항상 실행

spring.sql.init.mode = always

DDL SQL 파일 경로

spring.sql.init.schema-locations = classpath:order-schema.sql ...

or 명시적인 데이터 소스 구성 대신 JNDI.

=> Java 애플리케이션이 DataSource 같은 리소스를
코드에 직접 설정하지 않고 이름 찾아서 사용하는 방식.
리소스를 코드 밖에서 등록하고 필요할 때 이름 찾기.

5.1.3) 내장 서버 구성.

: server, port 속성 사용해 서블릿 컨테이너의 포트를 설정할 수 있음.

if) server.port 가 0이면?

=> 기본적으로 서버는 0번 포트 시작 X.

=> 대신, 사용 가능한 포트를 무작위로 선택하여 시작.

- HTTPS 요청 처리 위한 컨테이너 설정.

=> JDK: keytool 명령행 유틸리티 사용해서 keystore 생성.

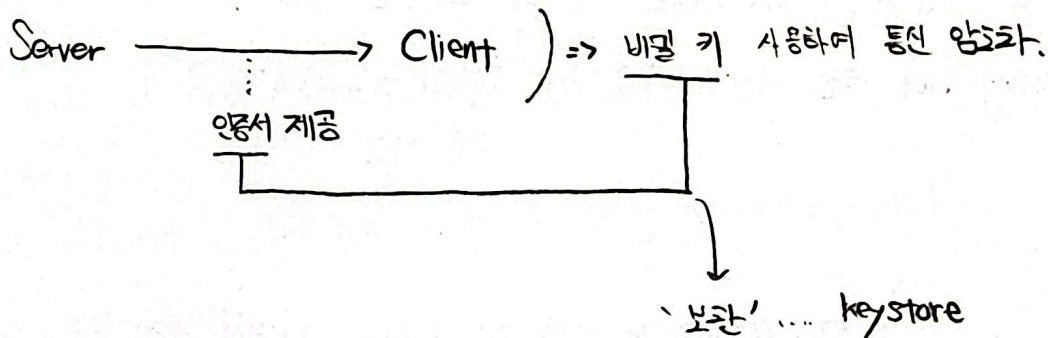
: Spring boot 애플리케이션에서 HTTPS 요청 처리하려면

SSL 인증서가 필요하고, 이를 key store 파일로 만들어야함.

: JDK에 기본 포함된 도구.

왜 필요?

=> HTTPS는 SSL/TLS 기반으로 동작.



설정 예시.

Server :

port: 8443 => 서버 실행할 포트

ssl :

key-store : file:///path/to/mykeys.jks (채널 파일. / 건너뛰면 file:///C:/mykeys.jks로 써야함)

key-store-password : letmein

key-password : letmein

↳ Java keyStore 파일

classpath : 스코프 resource 폴더 안에 파일 사용 O.

키스토어 내보내기 비밀번호에
접근하기 위한 비밀번호.

키스토어 파일 전체를 열기 위한 파일.

=> keytool로 키스토어 만들고 입력했던 비밀번호 설정

↳ 일반적으로 개발환경에서 key-store-password와 key-password 둘로 설정.

↳ 애플리케이션 JAR 파일에 키스토어 파일 넣는 경우는 classpath:를 URL로 지정.

5.1. 4) 로깅 구성.

=> 대부분의 애플리케이션은 로깅 제공.

Spring boot는 INFO 수준으로 콘솔에 로그 메시지를 쓰기 위해 logback 을 통해 로깅을 구성.

· 로깅 구성 제어' —————> classpath 내 root 에 logback.xml 과를 생성 가능
: src/main/resources

logback.xml 수정 시, 이러한 형태로 로그파일 제어가능.

But, 'spring boot 구성 속성' 사용 => logback.xml 생성안하고
변경 가능.

ex)

logging :

level :

root : WARN

org :

springframework :

security : DEBUG.

} 전체 애플리케이션 기본 로깅 레벨은 WARN 설정.

↳ WARN, ERROR 이상의 메시지만 출력.
INFO, DEBUG, TRACE 수준 출력X

} org.springframework.security 패키지
로그레벨만 DEBUG로 따로 설정.

if) /var/logs / 경로에 TacoCloud.log 파일에 수록.

=> logging.path, logging.file 속성 48.

logging :

path: /var/logs/ ✓

file: TacoCloud.log ✓

level :

root : WARN

org :

springframework :

security : DEBUG

→ 이 옵션 갖고 있다면 /var/logs/TacoCloud.log 에 수록
파일 크기인 10MB가 가득차면 새로운 로그파일을 생성.

로깅 레벨 종류 => TRACE, DEBUG, INFO, WARN, ERROR, OFF

⋮
가장낮게

⋮
디버깅에
필요한
정보

⋮
일반적인
실행
흐름

⋮
경고
상황

⋮
오류발생

⋮
비밀성하.

1.5) 다른 속성 값 가져오기!

String 과 숫자 값으로만 속성 값 설정X.

다른 속성으로부터 값 가져올 수 있음.

ex)

greeting.welcome 속성을 또 다른 속성인 spring.application.name 값으로 설정.

greeting:

welcome: $\boxed{\$ \{ \text{spring.application.name} \}}$ $\$ \{ \}$ 사용.

or 또 다른 Text에 포함 가능.

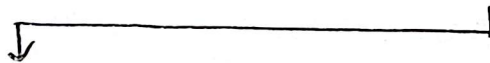
greeting:

welcome: You are using $\$ \{ \text{spring.application.name} \}$.

5.2 구성 속성 생성하기.

'구성 속성' => Bean의 속성.

스프링의 환경 구성으로부터 여러가지 구성 받아서 쓸게. => 구성 속성 사용 어떻게
Bean에 나타냄?



@ConfigurationProperties

: 이 어노테이션 지정하면, 해당 Bean의 속성들이
스프링의 환경 속성으로부터 주입.

5.2.1) 구성 속성 불러 쓰기.

① 구성 속성이 읽히.

-> src/main/resources 에 application.yml 생성.

- yml

taco:

orders:

page Size: 20

- properties

taco.orders.pageSize = 20.

실제로 ④ ConfigurationProperties 의서는 구성 데이터로 들어오 사용되는데 빈에 지정되는 경우가 많음

=> 구성 관련 정보 끼지.

pageSize = 속성 값 이 속성을 빌더 클래스로 주입 가능.

=> OrderProps 를 빌더 클래스로 사용.

Spring이 해당 클래스를 자동으로 찾아서 Bean으로 등록.
Bean으로 등록되면 나중에 자동으로 주입 받을 수 있음.

④ Component

④ ConfigurationProperties (prefix = "taco, orders")

④ Data

↳ application.yml or application.properties 에서 어떤 정크기 설정 값을 바인딩할 지 지정하는 정크기임.

```
public class OrderProps {  
    private int pageSize = 20;  
}
```

OrderProps 빈을 OrderController 이 주입.

④ Controller

④ Request Mapping ("/orders")

④ Session Attributes ("order")

```
public class OrderController {
```

```
    private OrderProps props;
```

```
    public OrderController (OrderRepository orderRepo, OrderProps props) {
```

```
        this.props = props;  
    }
```


만약, pageSize 속성의 값이 5~25 사이 값이 적용되고 싶다면?

L 코드 빈 있는 경우

: OrderController, pageSize 속성, pageSize 속성 사용하는 모든 클래스에
관하는 디노테이션 추가.

L 코드 빈 0.

: OrderProps 만 변경

:
@Validated

:
@Min (value=5, message=" ")

@Max (value=25, message=" ")

:
:

5.2.2) 구성 속성 메타산언.

만약, application.yml 에 taco.orders.pageSize 항목에 unknown property taco 라는
경고 메세지 뜬다면?

=> 속성에 관한 메타 데이터 없어서 그렇다.

구성 속성 메타 데이터 없이도 동작하는데 문제 X.

But, 있다면 구성 속성에 관해 최소한의 정보 제공해주므로 유용.

- 의존성 추가.

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId>

spring-boot-configuration-processor

</artifactId>

<optional> true</optional>

</dependency>

1)

spring-boot-configuration-processor 은

① ConfigurationProperties 애노테이션이 지정된 애플리케이션 클래스에 관한
메타 데이터를 생성하는 애노테이션 처리기.

생성된 메타 데이터 => application.yml or application.properties
작성시 자동-관성 기능 제공 및 속성 문서 보이기.

2)

커스텀 구성 속성에 관한 메타 데이터 생성.

=> /src/main/resources/META-INF 에 additional-spring-configuration-metadata.json

폴더 생성 등 →

파일 생성

...

type에는 반드시
Java 클래스 이름 적어야함.

```
{
  "properties": [
    {
      "name": "taco.orders.page-size",
      "type": "java.lang.Integer",
      "description": " "
    }
  ]
}
```


3) 프로파일 사용해서 구성.

애플리케이션이 서로 다른 런타임 환경에 배포, 설치될 때 구성 명세가 달라짐.

ex)

DB 연결 명세가 개발환경과 다름.

개발시점 : 자동-구성된 내장 H2 데이터 베이스 사용.

But, 프로덕션 환경 : 환경변수로 DB 구성 속성 설정.

↳ 하나 이상의 구성 속성을 환경변수로 저장한 번거롭.

↳ 환경 변수의 변경을 추적/관리 어렵.

=> 스프링 프로파일 사용.

↳ 런타임 시에 활성화되는 프로파일에 따라 서로 다른 빈, 구성 클래스, 구성 속성들이 적용 아 유지되도록 하는 것이 프로파일.

ex)

개발 / 디버깅 목적으로 내장 H2 DB 사용,

타고 클라이언트 로깅 수준 DEBUG.

프로덕션 환경

: 외부 MySQL DB 사용 + 로깅수준 WARN 설정.

↳ application.yml 을 변경하기 않고 배포 및 설치 시

DEBUG 로깅 수준 / 내장 H2 DB 여전히 사용.

∴ 이럴 때 프로덕션 환경에 적합한 속성들을 설정하는 프로파일 정의

5.3.1) 프로파일 특징 속성 정리

프로파일이 특정한 속성 정리는 방법.

- ① 프로젝트 환경에 속성들만 포함하는 파더 .yml or .properties 생성.
- ② YAML 구성에서만 가능한 방법으로 정리.

①

⇒ 이때 파일을 생성 시, 규칙이 존재.

L application - 프로파일 이름. yml

L application - 프로파일 이름. properties) ex ⇒ application - prod. yml

spring:

datasource:

uri: ~~~~~

username: ~

password: ~

logging:

level:

tags: WARN

②

: 하이픈(---) 추가.

logging:

level:

tags: DEBUG

spring:

profiles: prod

datasource:

uri: ~~~~~

.....

spring.profiles 값 지정

특정 속성 설정은 prod 프로파일에만 적용.

5.3.2) 프로파일 활성화.

: 프로파일 특정 속성들이 설정은 해당 프로파일이 활성화 되어야함.

How?

→ `spring.profiles.active` 속성이 지정.

ex)

```
spring:
  profiles:
    active:
      - prod.
```

: 좀더 많은 프로파일 활성화 방법.

1) .yml에서 적용

: 기본 프로파일이 된다.

: 환경 변수를 사용해서 활성화 프로파일을 설정하는 것은 가능하다.

↳ `% export SPRING_PROFILES_ACTIVE=prod`

: 해당 컴퓨터에 배포되는 어떤 애플리케이션에서도 prod 프로파일이 활성화된다.

`spring.profiles.active` 속성은 여러 개의 프로파일이 포함될 수 있다.

- 환경 변수 사용해서 프로파일 활성화 할 때는 쉼표 사용.

↳ `% export SPRING_PROFILES_ACTIVE=prod,audit,ha`

↓ ... YAML

```
spring:
  profiles:
    active:
      - prod
      - audit
      - ha
```

5.3.3) 프로파일을 사용해서 조건별로 빈 생성.

: 서로 다른 프로파일에서 각각에 적합한 빈들을 제공하는 것이 필요할 때 있다.

일반적으로 자바 구성 클래스에 선언된 빈은 활성화되는 프로파일과는 무관하게 생성.

ex)

@Configuration

```
public class AppConfig() {
```

@Bean

```
public MyService myService {
```

```
    ...
```

```
}
```

```
}
```

i) 일반적인 경우

application.properties에

spring.profiles.active=dev 가 되어있을 때

myService()는 항상 빈으로 등록.

... @Profile 사용.

@Configuration

@Profile("prod")

```
public class ProdConfig {
```

@Bean

```
public MyService myService {
```

```
    ...
```

```
}
```

```
}
```

ii) 프로파일 지정된 경우.

spring.profiles.active=prod 일 때만 작동.

prod 프로파일 아니면 빈으로 등록 x.

@Configuration + @Bean => 기본적으로 모든 환경에서 빈으로 등록.

@profile("dev") => 해당 프로파일을 때만 등록.

- dev 프로파일 or qa 프로파일 중 하나 활성화 될 때.

@ Bean

@ Profile ({"dev", "qa"})

public MyService myService () {

:

}

- prod 프로파일 활성화 x 될 때.

@ Bean

@ Profile ("!prod")

public MyService myService () {

:

}

- @Profile & @Configuration 이 지정된 클래스 전체에 사용 가능.

@ Profile ({"!prod", "!qa"})

@ Configuration

public class DevelopmentConfig {

@ Bean

public MyService myService () {

:

}

}

=> prod, qa 프로파일 모두 활성화 x 될 때 MyService 빈이 생성.